# Data Scientist Interview Study Notes

# Data Scientist Interview Study Notes

**Who this is for:** Anyone preparing for a Data Scientist interview, especially for roles that cover customer solutions in IT, manufacturing, finance and services, machine data analysis, signal processing, and imaging (MRI/CT, optical flow).

**How to read this:** Each chapter follows the same pattern: 1. **Big idea**: what it is, in plain words. 2. **Math**: only what you need, explained step by step. 3. **Example**: a small worked example with numbers you can check by hand. 4. **Interview tip**: what interviewers usually ask, and how to answer. 5. **Check yourself**: a few questions to test your understanding.

Work through the chapters in order. The early chapters (math and statistics) make the later ones easier.

---

# Part 1: Math You Need

## Chapter 1: Basic Statistics Words

### Big idea

Statistics is a way of describing a pile of numbers, and deciding how much to trust what you see.

### Mean, median, mode

- **Mean (average):** add everything and divide by the count.
- **Median:** the middle value when sorted. Better than the mean when there are extreme values.
- **Mode:** the most common value.

**Example:** Monthly salaries (in thousands): 20, 22, 22, 25, 200

- Mean = (20 + 22 + 22 + 25 + 200) / 5 = 289 / 5 = **57.8**
- Median = **22** (the middle value)

The mean is pulled up by the single 200. That's why salary data is usually reported with the median.

### Variance and standard deviation

These measure how spread out the numbers are.

```
Mean:                μ = (1/n) × Σ xᵢ
Variance:            σ² = (1/n) × Σ (xᵢ − μ)²
Standard deviation:  σ = √(variance)
```

Read it like this: for each number, find how far it is from the mean, square that distance (so negatives don't cancel), average them, and take the square root to get back to the original units.

**Example:** Data = 2, 4, 4, 4, 5, 5, 7, 9 - Mean μ = 40 / 8 = **5** - Squared distances: 9, 1, 1, 1, 0, 0, 4, 16. Sum = 32 - Variance = 32 / 8 = **4** - Standard deviation = √4 = **2**

### Interview tip

If asked "what's the difference between variance and standard deviation?", say: "Variance is in squared units, which is hard to interpret. Standard deviation is back in the original units, so it's easier to explain."

### Check yourself

1. Why is the median often better than the mean for house prices?
2. For data 10, 10, 10, what is the standard deviation? (Answer: 0. No spread.)

---

# Chapter 2: Probability

### Big idea

Probability is a number between 0 and 1 that tells you how likely something is. 0 means impossible, 1 means certain.

### Core rules

- **Addition rule:** P(A or B) = P(A) + P(B) − P(A and B)
- **Multiplication rule (independent events):** P(A and B) = P(A) × P(B)
- **Conditional probability:** P(A | B) means "probability of A, given that B has happened."

```
P(A | B) = P(A and B) / P(B)
```

**Example:** A deck has 52 cards. P(King) = 4/52. P(King | card is a face card) = 4/12 (there are 12 face cards, and 4 of them are kings).

### Bayes' theorem

This is one of the most important formulas in data science. It lets you flip a conditional probability around.

```
P(A | B) = P(B | A) × P(A) / P(B)
```

**Example: Medical test** - A disease affects **1%** of people. - The test catches **90%** of sick people (this is the sensitivity). - The test wrongly flags **5%** of healthy people (false positive rate). - You test positive. What's the chance you're actually sick?

Let's count it using 10,000 people: - Sick people: 1% of 10,000 = 100. Test catches 90% → 90 true positives. - Healthy people: 9,900. Test wrongly flags 5% → 495 false positives. - Total positives = 90 + 495 = 585. - P(sick | positive) = 90 / 585 ≈ **15.4%**

Even with a good test, most positive results are false, because the disease is rare. This is called the **base rate** effect, and interviewers love it.

### Common distributions

- **Normal (bell curve):** Many natural things (heights, measurement errors) follow it. About 68% of values fall within 1 standard deviation of the mean, 95% within 2, and 99.7% within 3.
- **Binomial:** Number of successes in n yes/no trials. Example: number of defective items in a batch of 100.
- **Poisson:** Count of events in a fixed time. Example: number of machine breakdowns per week.

### Interview tip

Explain Bayes with a simple example, like the medical test above. Don't start with the formula.

### Check yourself

1. In a spam filter, if 40% of all emails are spam, what's the prior probability of spam? (Answer: 0.4)
2. Why does a rare disease make a positive test less reliable?

---

# Chapter 3: Calculus You Need (Just Enough for Machine Learning)

### Big idea

A **derivative** tells you how fast something changes. In machine learning, we use it to find the direction to move to make an error smaller.

### Derivative

For a function f(x), the derivative f'(x) is the slope at point x.

**Example:** $f(x) = x^2$ - $f'(x) = 2x$ - At x = 3, the slope is 6. This means if you increase x a little, f goes up about 6 times as fast.

### Partial derivative and gradient

When a function has many inputs (like many model weights), we take a derivative for each one. The list of all of them is the **gradient**.

**Example:** $f(w_1, w_2) = w_1^2 + 3w_2^2$ - $\partial f/\partial w_1 = 2w_1$ - $\partial f/\partial w_2 = 6w_2$ - Gradient = $[2w_1, 6w_2]$

### Chain rule

If y depends on u, and u depends on x, then:

```
dy/dx = (dy/du) × (du/dx)
```

This is the engine behind backpropagation in neural networks.

### Interview tip

You don't need to derive complex formulas on a whiteboard. Be ready to say: "The gradient points uphill. We move the opposite way, to reduce the error."

### Check yourself

1. What is the derivative of $f(x) = 5x^3$? (Answer: $15x^2$)
2. In which direction should you move to reduce a loss function?

---

## Chapter 4: Linear Algebra You Need

### Big idea

Linear algebra is the language of data. A data row is a **vector**, a whole table is a **matrix**, and most ML operations are matrix multiplications.

### Vectors and dot product

A vector is a list of numbers. The dot product multiplies matching entries and adds them up.

```
a = [1, 2, 3]
b = [4, 5, 6]
a · b = (1×4) + (2×5) + (3×6) = 4 + 10 + 18 = 32
```

In ML, a model's prediction is often a dot product of weights and features: prediction = $w \cdot x + b$.

### Distance

- **Euclidean distance** (straight-line): between (1, 2) and (4, 6)

  ```
  d = √((4−1)² + (6−2)²) = √(9 + 16) = √25 = 5
  ```

- **Manhattan distance** (city blocks): $|4-1| + |6-2| = 3 + 4 = 7$

### Matrices

A matrix is a table of numbers. A dataset with 100 rows and 5 columns is a 100 × 5 matrix.

### Eigenvalues and eigenvectors (used in PCA)

An eigenvector of a matrix is a direction that doesn't change when the matrix is applied to it. It only gets stretched. The stretch factor is the eigenvalue.

Why it matters: in PCA, the eigenvectors of the data's covariance matrix point in the directions of greatest spread. The eigenvalues tell you how much spread is in each direction.

### Interview tip

If asked "what is an eigenvector?", say: "It's a direction where a matrix only stretches things, without rotating them. In PCA, these are the directions where the data varies most."

### Check yourself

1. Compute the dot product of [2, 0] and [3, 4]. (Answer: 6)
2. Why is Euclidean distance affected by feature scaling? (Answer: a feature with large numbers dominates the distance.)

---

# Part 2: Statistics for Hypothesis Testing

## Chapter 5: Sampling and the Central Limit Theorem

### Big idea

We usually can't measure everyone. We take a **sample** and use it to guess about the whole **population**.

### Central Limit Theorem (CLT)

If you take many samples and compute each sample's mean, those means will form a roughly normal (bell) curve, even if the original data isn't normal. The bigger each sample, the better this works.

Why this matters: it's what lets us make confidence statements about averages.

### Standard error

The standard error (SE) is how much a sample mean is expected to vary from sample to sample.

```
SE = σ / √n
```

where σ is the standard deviation and n is the sample size. Bigger samples give smaller SE.

**Example:** σ = 8, n = 64. SE = 8 / √64 = 8 / 8 = **1**.

### Confidence interval

A range that likely contains the true value.

```
95% CI = sample mean ± 1.96 × SE
```

**Example:** Sample mean = 52, SE = 1. 95% CI = 52 ± 1.96 × 1 = **[50.04, 53.96]**

Correct wording: "We are 95% confident that the true mean is between 50.04 and 53.96." This means the method captures the true value 95% of the time when repeated. It does not mean there's a 95% chance the true value is in this specific range.

### Interview tip

Interviewers often test whether you understand the wording. Use the "repeated sampling" explanation above.

### Check yourself

1. If the sample size quadruples, what happens to the standard error? (Answer: it halves, because SE = σ/√n.)

---

## Chapter 6: Hypothesis Testing

### Big idea

We start by assuming nothing special is happening (the **null hypothesis, $H_0$**). Then we check whether the data is so surprising under that assumption that we reject it.

### Key terms

- **Null hypothesis ($H_0$):** no effect, no difference. Example: "The new machine setting doesn't change defect rate."
- **Alternative hypothesis ($H_1$):** there is an effect.
- **p-value:** the probability of seeing data at least this extreme, assuming $H_0$ is true.

- **Significance level (α):** the threshold, usually 0.05. If p < α, we reject $H_0$.

**Important:** A p-value is NOT the probability that $H_0$ is true. Say this clearly in interviews.

### Type I and Type II errors

| | **$H_0$ is actually true** | **$H_0$ is actually false** |
|---|---|---|
| **We reject $H_0$** | Type I error (false alarm), chance = α | Correct |
| **We don't reject $H_0$** | Correct | Type II error (missed effect) |

Analogy: A smoke alarm that goes off when there's no fire is a Type I error. One that stays silent during a real fire is a Type II error.

### t-test (comparing a mean to a value)

```
t = (sample mean − hypothesized mean) / (s / √n)
```

where s is the sample standard deviation.

**Example:** A factory claims bolts are 50 mm long. A sample of 64 bolts has mean 52 mm and standard deviation 8 mm. - t = (52 − 50) / (8 / √64) = 2 / 1 = **2** - A t-value around 2 with this sample size gives a p-value of about **0.05**, which is borderline significant.

### Chi-square test (for categories)

Used when you have counts in categories, for example "does defect type depend on the shift?"

```
χ² = Σ (Observed − Expected)² / Expected
```

**Example:** Expected 50 defects per shift, observed 60. Contribution = $(60 - 50)^2 / 50 = 100 / 50 = 2$. Add the contributions from all categories, then compare to a chi-square table.

### ANOVA (comparing 3 or more means)

If you compare three or more group means at once, don't run many t-tests (this inflates false alarms). Use ANOVA instead. It asks: "Is the variation between groups bigger than the variation within groups?"

### Correlation vs. causation

- **Correlation:** two things move together. Measured by the correlation coefficient r, from −1 to +1.
- **Causation:** one thing causes the other. Needs experiments or careful study design.

**Example:** Ice cream sales and drowning rates are correlated. Hot weather causes both. Ice cream doesn't cause drowning.

### Interview tip

If asked "what's a p-value?", say: "It's the chance of seeing data this extreme if nothing is really happening. Small p-value means the data would be surprising under the null hypothesis."

### Check yourself

1. A p-value is 0.03 and α = 0.05. What do you conclude? (Answer: reject $H_0$.)
2. Which error is it when you conclude a new process works, but it actually doesn't? (Answer: Type I.)

---

## Chapter 7: A/B Testing (Forming and Testing a Hypothesis for a Business)

### Big idea

An A/B test compares two versions (control A and treatment B) on random groups to see which performs better.

### Steps

1. State the hypothesis: "New checkout button increases conversion."
2. Pick a metric: conversion rate.

3. Decide the sample size before the test starts.
4. Randomly split users.
5. Run the test, then check whether the difference is statistically significant.

### Example

- Control A: 100 conversions out of 1,000 visitors (10%).
- Treatment B: 120 conversions out of 1,000 visitors (12%).

Pooled rate = (100 + 120) / 2000 = 0.11

```
SE = √( 0.11 × 0.89 × (1/1000 + 1/1000) ) = √(0.0979 × 0.002) ≈ 0.0140
z = (0.12 − 0.10) / 0.0140 ≈ 1.43
```

A z of 1.43 gives a two-sided p-value of about **0.15**. That's above 0.05, so the difference is **not** statistically significant. The result is that we don't have enough evidence yet, not that B failed.

### Interview tip

Mention these points: sample size planning, avoiding "peeking" at results too early, and checking for practical significance (not only statistical significance).

### Check yourself

1. Why must you decide the sample size before running the test? (Answer: stopping early when results look good inflates false positives.)

---

# Part 3: Machine Learning Basics

## Chapter 8: Machine Learning Fundamentals

### 8.1 What machine learning really is

Traditional programming means you write the rules yourself: "if temperature > 80, flag the machine." Machine learning flips this. You give the computer examples (inputs and, usually, the correct answers), and the computer figures out the rules by itself. The "rules" are stored as numbers called **parameters** or **weights**. Training is the process of adjusting those numbers until the model's predictions match the examples well.

A simple way to explain it in an interview:

> "Instead of hand-coding the rule for detecting a faulty machine, I give the model thousands of past sensor readings labelled as 'faulty' or 'healthy'. The model finds the pattern that separates them, and I check how well that pattern works on readings it has never seen."

### 8.2 The machine learning workflow (end to end)

Interviewers like candidates who think in the full lifecycle, not just the model. The typical steps are:

1. **Frame the problem.** What decision will the prediction support? Who uses it? What does a wrong prediction cost?
2. **Collect and understand the data.** Where does it come from, how often is it updated, is there a label, and can we trust it?
3. **Clean and prepare the data.** Handle missing values, duplicates, wrong units, and outliers (Chapter 19).
4. **Split the data.** Train, validation, and test sets, split in a way that matches how the model will be used (Section 8.5).
5. **Choose a baseline.** A simple model, like predicting the most common class or the previous value. Every fancy model must beat this.
6. **Train candidate models.** Try a few algorithms, tune their hyperparameters on the validation set.
7. **Evaluate honestly.** Use the test set once, with metrics that match the business cost (Chapter 9).
8. **Explain and check the model.** Are the important features sensible? Are there fairness or bias concerns?
9. **Deploy.** Serve predictions through an API or batch job.
10. **Monitor.** Watch for data drift (inputs change) and concept drift (the relationship between inputs and output changes). Retrain when needed.

**Interview tip:** If asked “walk me through a project,” follow this order. It shows you understand the whole process, not only the algorithm.

### 8.3 Types of learning

**Supervised learning** uses labelled data. The model learns a mapping from inputs X to a known output y. - **Classification:** the output is a category. Examples: spam or not spam, defective or not defective, which customer segment. - **Regression:** the output is a continuous number. Examples: house price, expected machine temperature in 10 minutes, sales next month.

**Unsupervised learning** uses unlabelled data. The model looks for structure. - **Clustering:** group similar items (customer segments, machine operating modes). - **Dimensionality reduction:** compress many features into fewer ones (PCA, Chapter 17). - **Anomaly detection:** find items that look unlike the rest.

**Semi-supervised learning:** a little labelled data plus a lot of unlabelled data. Useful when labelling is expensive, such as medical images labelled by specialists.

**Reinforcement learning:** an agent takes actions in an environment and learns from rewards. Less common in typical data science roles, but good to know the idea.

**Interview tip:** A common opening question is “What’s the difference between classification and regression?” Answer: “The output type. Classification predicts a category, regression predicts a number. The metrics differ too: accuracy or F1 for classification, MAE or RMSE for regression.”

### 8.4 Features, labels, and the feature matrix

- **Feature (X):** an input variable. Example: temperature, vibration, age, income.
- **Label or target (y):** the value you want to predict.
- **Sample or observation:** one row of data.
- **Feature matrix:** all samples and features in a table, usually shape (n_samples, n_features).

Good features matter more than the choice of algorithm in most real projects. A simple model with well-designed features often beats a complex model with raw features.

### 8.5 Train, validation, and test sets

- **Training set (about 60–80%):** the model learns its parameters from this.
- **Validation set (about 10–20%):** used to compare models and tune hyperparameters.
- **Test set (about 10–20%):** used once, at the very end, to report honest performance.

**Why three sets?** If you tune the model by looking at the test score repeatedly, you are indirectly fitting the test set, and your reported score will be too optimistic.

**Time-based data:** For time series or machine data recorded over time, split by time. Train on the earlier period, validate on the next, test on the most recent. Random splitting lets the model peek at the future.

**Interview tip:** “How did you split your data?” A strong answer names the split ratio, says whether it was random, stratified (keeps class proportions), or time-based, and explains why.

### 8.6 Overfitting and underfitting

Think of a student preparing for an exam: - **Underfitting:** the student barely studied. Poor on practice tests and the real exam. - **Good fit:** the student understood the concepts. Good on both. - **Overfitting:** the student memorised the answers to practice questions. Perfect on practice, poor on new questions.

In numbers: - Underfitting: low training score, low validation score. - Overfitting: very high training score, noticeably lower validation score.

**Causes of overfitting:** too complex a model, too few samples, too many features, training for too long (in neural networks), and noisy labels.

**Remedies:** - Get more data. - Simplify the model (fewer layers, shallower trees). - Regularization (L1, L2, dropout). - Early stopping. - Cross-validation to detect it. - Remove irrelevant or duplicate features.

### 8.7 Bias-variance tradeoff

- **Bias:** error from wrong or too simple assumptions. A straight line fitted to a curved pattern has high bias.

- **Variance:** error from being too sensitive to the particular training data. A very deep tree trained on a small sample has high variance: change the sample a little and the tree changes a lot.
- **Irreducible error:** noise in the data that no model can remove.

Expected prediction error roughly equals:

```
Expected error ≈ Bias² + Variance + Irreducible noise
```

Simple models usually have high bias and low variance. Complex models usually have low bias and high variance. The goal is the sweet spot in the middle.

**Interview tip:** Explain it with a dartboard. High bias: darts clustered in one spot, but far from the bullseye. High variance: darts spread all over the board. Good model: darts close to the bullseye and close together.

### 8.8 Cross-validation

Instead of one train/validation split, split the training data into k folds (usually k = 5 or 10). Train on k−1 folds, validate on the remaining one. Repeat so each fold is the validation fold once. Average the k scores.

**Why it helps:** a single split can be lucky or unlucky. Cross-validation gives a more stable estimate and uses all data for both training and validation.

**Variants:** - **Stratified k-fold:** keeps the class ratio the same in each fold. Use this for classification, especially when classes are imbalanced. - **Time series split:** each fold trains on the past and validates on the next block. No shuffling. - **Group k-fold:** keeps all rows from the same group (for example, the same patient or the same machine) in one fold, to avoid leakage.

### 8.9 Hyperparameters and tuning

- **Parameters:** learned from data during training (weights, tree split thresholds).
- **Hyperparameters:** set before training (tree depth, k in k-NN, learning rate, regularization strength).

Tuning methods: - **Grid search:** try every combination in a list. Thorough but expensive. - **Random search:** sample combinations at random. Often finds good settings faster, because only a few hyperparameters usually matter a lot. - **Bayesian optimisation** (for example, Optuna): uses past results to choose the next trial. Efficient for expensive models.

Always tune on the validation set or inside cross-validation, never on the test set.

### 8.10 Data leakage

Leakage means information that would not be available at prediction time sneaks into training. It makes validation scores look excellent, and then the model fails in production.

Common forms: - **Preprocessing before splitting:** fitting a scaler or imputer on the full dataset. - **Target leakage:** a feature that is only known after the event. Example: “number of repair orders created” when predicting whether a machine will fail. Repair orders are created because of the failure. - **Time leakage:** using future data to predict the past, for example through random shuffling of time series. - **Duplicate rows** across train and test.

**Interview tip:** “How do you prevent leakage?” Answer: “I split first, fit all preprocessing on the training data only, check each feature for whether it would be known at prediction time, and keep time order for time-based data.”

### 8.11 Rapid interview questions for this chapter

**Q: What is the difference between a parameter and a hyperparameter?** A parameter is learned from data (for example, the weights of a linear model). A hyperparameter is chosen by you before training (for example, the number of trees in a random forest). You tune hyperparameters using validation performance.

**Q: Why do we need a baseline model?** A baseline tells you whether a complex model adds real value. If predicting the majority class already gives 95% accuracy, a model at 96% is not necessarily useful. Simple baselines include the majority class, the mean, or the last observed value.

**Q: How would you tell if a model has high variance?** Its training performance is much better than its validation performance, and the gap changes a lot when I change the training data. Remedies include more data, regularization, simpler models, or ensembling.

**Q: What is concept drift?** When the relationship between inputs and the target changes over time. Example: a customer churn model trained before a price increase may stop working after it. Detect it by monitoring prediction quality and input distributions, then retrain.

---

## Chapter 9: Evaluation Metrics in Depth

### 9.1 Why metrics matter

A metric is how you define “good” for your model. Choosing the wrong metric leads to a model that looks great on paper and fails in practice. The right metric depends on the cost of each type of mistake.

### 9.2 Confusion matrix

For a binary classifier, every prediction falls into one of four boxes:

| | **Predicted Positive** | **Predicted Negative** |
|---|---|---|
| **Actually Positive** | True Positive (TP) | False Negative (FN) |
| **Actually Negative** | False Positive (FP) | True Negative (TN) |

- **TP:** a faulty machine correctly flagged.
- **FN:** a faulty machine missed. Often the most expensive error in maintenance.
- **FP:** a healthy machine flagged. Costs an unnecessary inspection.
- **TN:** a healthy machine correctly left alone.

### 9.3 Accuracy, precision, recall, F1

- **Accuracy** = (TP + TN) / all. Simple, but misleading with imbalanced data.
- **Precision** = TP / (TP + FP). Of everything flagged, how much was correct. High precision means few false alarms.
- **Recall (sensitivity, true positive rate)** = TP / (TP + FN). Of all real positives, how many were caught. High recall means few misses.
- **F1 score** = harmonic mean of precision and recall = 2PR / (P + R). Useful when you need a single number that balances both. The harmonic mean punishes a very low value in either one.
- **Specificity** = TN / (TN + FP). Of the real negatives, how many were correctly left alone.

**Worked example:** TP = 40, FP = 10, FN = 20, TN = 30. - Accuracy = 70 / 100 = 0.70 - Precision = 40 / 50 = 0.80 - Recall = 40 / 60 = 0.67 - F1 = 2 × 0.80 × 0.67 / (0.80 + 0.67) ≈ 0.73

### 9.4 Choosing the metric by business cost

| **Situation** | **Costly mistake** | **Prioritise** |
|---|---|---|
| Fault detection on a production line | Missing a fault leads to a line breakdown | Recall |
| Spam filter that deletes mail | Deleting an important email | Precision |
| Fraud detection | Both missing fraud and blocking real customers hurt | F1, or PR-AUC |
| Credit approval | Approving a bad loan | Precision of the “approve” class, plus business cost analysis |
| Balanced classes, both errors matter equally | — | Accuracy or balanced accuracy |

**Interview tip:** Always link the metric to a cost. “Missing a bearing failure costs roughly ten times an unnecessary inspection, so I would set the threshold to favour recall and report the precision trade-off to the maintenance team.”

### 9.5 Threshold and the precision-recall tradeoff

Most classifiers output a probability or score. You choose a threshold (default 0.5) to turn it into a class. - Lower threshold: more positives predicted, recall goes up, precision goes down. - Higher threshold: fewer positives predicted, precision goes up, recall goes down.

Plot precision and recall for every threshold to see the trade-off. Then pick the threshold that matches the business cost, not the default 0.5.

### 9.6 ROC curve and AUC

- **ROC curve:** plots True Positive Rate (recall) against False Positive Rate (FP / (FP + TN)) for every threshold.
- **AUC (area under the ROC curve):** a single number. 0.5 means random guessing, 1.0 means perfect ranking.

**Meaning of AUC:** the probability that a randomly chosen positive example gets a higher score than a randomly chosen negative example.

**Strength:** it measures ranking quality and does not depend on one threshold. **Weakness:** on heavily imbalanced data, ROC-AUC can look good even when precision is poor, because the large number of true negatives keeps the false positive rate small. In that case, use the Precision-Recall curve and PR-AUC.

### 9.7 Multi-class metrics

With more than two classes, compute precision and recall per class, then combine: - **Macro average:** average the per-class scores, treating every class equally. Good when rare classes matter. - **Micro average:** pool all TP, FP, FN across classes, then compute. Dominated by frequent classes. - **Weighted average:** average per-class scores weighted by class size.

Also look at the **confusion matrix** for multi-class problems. It shows which classes get confused with each other, which the summary numbers hide.

### 9.8 Probability calibration

A model is well calibrated if predictions of 0.8 are correct about 80% of the time. Some models (like random forests and naive Bayes) give poorly calibrated probabilities even with good ranking. Calibration plots and methods like Platt scaling or isotonic regression fix this. This matters when the probability itself drives a decision, such as pricing or risk scores.

### 9.9 Regression metrics

- **MAE (mean absolute error):** average of |actual − predicted|. Easy to explain, in the original units, robust to outliers.
- **MSE (mean squared error):** average of (actual − predicted)². Punishes large errors heavily.
- **RMSE:** square root of MSE. Back in the original units, and still punishes large errors.
- **MAPE:** average of |error| / |actual| as a percentage. Easy to communicate, breaks when actual values are near zero.
- **$R^2$ (coefficient of determination):** 1 − (residual sum of squares / total sum of squares). Share of variance explained. 1 is perfect, 0 means no better than predicting the mean. It can be negative for a bad model.

**Worked example:** actual = [10, 12, 15], predicted = [11, 12, 13]. - Errors: 1, 0, −2 - MAE = (1 + 0 + 2) / 3 = 1.0 - MSE = (1 + 0 + 4) / 3 = 1.67 - RMSE = √1.67 ≈ 1.29

### 9.10 Interview questions for this chapter

**Q: Your model has 98% accuracy. Is it good?** Not necessarily. If only 2% of cases are positive, a model that always predicts negative also gets 98%. Check precision, recall, and the confusion matrix. Compare with a majority-class baseline.

**Q: When would you prefer PR-AUC over ROC-AUC?** When the positive class is rare and I care about the quality of positive predictions, such as fraud or rare machine faults.

**Q: What does an F1 of 0.5 tell you?** It indicates a weak balance between precision and recall. I would look at which of the two is low and decide, from the business cost, whether to adjust the threshold or the model.

**Q: Why is RMSE larger than MAE for the same predictions?** Squaring errors gives extra weight to large errors before the average is taken. A large RMSE compared with MAE signals that a few big errors dominate.

**Q: How do you choose a threshold?** Use the validation set. Plot precision and recall (or cost) against thresholds, choose the point that matches the business cost, then confirm on the test set once.

---

# Part 4: Algorithms Named in the JD

## Chapter 10: Linear Regression

### 10.1 The idea

Linear regression assumes the target is roughly a weighted sum of the features, plus a constant. It is the simplest and most interpretable regression model, and it is the base for many other ideas.

```
ŷ = b + w₁x₁ + w₂x₂ + ... + wₚxₚ
```

- Each **weight** $w_i$ says: "if feature $x_i$ increases by 1 unit, and all other features stay the same, the prediction changes by $w_i$."
- **Bias** b is the prediction when all features are zero.

**Example:** Predict monthly machine energy use (kWh) from production hours and ambient temperature. A fitted model: $\hat{y} = 20 + 3.5 \times \text{hours} + 0.8 \times \text{temperature}$. A 1-hour increase in production adds 3.5 kWh, holding temperature fixed.

### 10.2 How it learns: ordinary least squares

The model chooses weights that minimise the sum of squared residuals (residual = actual − predicted):

```
Loss = Σ (yᵢ − ŷᵢ)²
```

There are two common ways to solve it: 1. **Closed-form (normal equation):** $\theta = (X^TX)^{-1} X^Ty$. Exact, but requires inverting a matrix, which is slow when there are many features. 2. **Gradient descent:** start with random weights, repeatedly move them against the gradient of the loss. Scales better to large data.

Scikit-learn's `LinearRegression` uses a least-squares solver. For very large data, `SGDRegressor` uses stochastic gradient descent.

### 10.3 Assumptions (and what breaks when they fail)

Interviewers often ask about these: 1. **Linearity:** the target is roughly linear in the features. Check with residual plots. 2. **Independence:** errors are independent of each other. Violated in time series, where errors are often correlated over time. 3. **Homoscedasticity:** errors have constant variance. If the spread grows with the prediction (a funnel shape in the residual plot), predictions have unreliable confidence intervals. Log-transforming the target can help. 4. **Normality of errors:** needed for exact confidence intervals and tests on small samples. For prediction alone, it matters less. 5. **No (perfect) multicollinearity:** features should not be almost linear combinations of each other.

Note: linear regression does not require the features themselves to be normally distributed.

### 10.4 Multicollinearity

When two features carry almost the same information (for example, "temperature in Celsius" and "temperature in Fahrenheit"), the weights become unstable. Small changes in the data can flip their signs, even though the predictions stay fine.

**Detection:** correlation matrix, and the **Variance Inflation Factor (VIF)**:

```
VIF_j = 1 / (1 − R²_j)
```

where $R^2_j$ comes from regressing feature j on all the other features. A VIF above about 5 to 10 is a warning sign.

**What to do:** drop one of the correlated features, combine them, use PCA, or use Ridge regression.

**Important:** multicollinearity hurts interpretation of coefficients, but it does not necessarily hurt prediction.

### 10.5 Regularization: Ridge, Lasso, Elastic Net

Regularization adds a penalty for large weights, so the model prefers simpler solutions.

- **Ridge (L2):** loss = $\Sigma(y - \hat{y})^2 + \lambda \Sigma w^2$. Shrinks all weights toward zero, but rarely to exactly zero. Handles multicollinearity well.
- **Lasso (L1):** loss = $\Sigma(y - \hat{y})^2 + \lambda \Sigma |w|$. Can set some weights exactly to zero, so it also performs feature selection.
- **Elastic Net:** a mix of both, controlled by a ratio parameter.

λ (alpha in scikit-learn) controls strength. Large λ means strong shrinkage, which risks underfitting. Choose λ with cross-validation.

**Important:** always standardise features before regularization. Otherwise, features measured in large units get a smaller effective penalty.

### 10.6 Interpreting the model

- Look at the **sign** (direction) and **size** of each standardised coefficient to compare importance.

- Report **confidence intervals** for coefficients if the audience needs statistical certainty (from statsmodels).
- Check **residuals**: they should look random around zero.
- Use **$R^2$** and **adjusted $R^2$** (which penalises adding useless features) to judge fit.

### 10.7 Interview questions for this chapter

**Q: When would you use linear regression over a complex model?** When interpretability matters (for example, explaining to a plant manager why energy rose), when the data is small, or as a strong baseline. Complex models must beat it to be worth their cost.

**Q: What happens to the coefficients if you add a highly correlated feature?** They become unstable and can change sign, and the standard errors inflate. Predictions may stay similar. I'd check VIF and consider removing or combining features.

**Q: Why do we standardise before Ridge or Lasso?** The penalty is based on the size of the weights, and a weight's size depends on its feature's units. Standardising makes the penalty fair across features.

**Q: Ridge or Lasso?** Lasso if I want automatic feature selection and suspect many features are useless. Ridge if I expect many small useful effects or features are correlated. Elastic Net when I want a bit of both.

**Q: Your residual plot shows a funnel shape. What does it mean?** Heteroscedasticity: the error variance changes with the prediction. Predictions are still usable on average, but intervals are wrong. Try a log transform of the target or a model that handles varying variance.

**Q: What is $R^2$ and what's its limitation?** $R^2$ is the share of variance explained. It always increases when you add a feature, even a random one, so use adjusted $R^2$ or cross-validated error to compare models with different numbers of features.

---

## Chapter 11: Logistic Regression

### 11.1 The idea

Logistic regression is a **classification** model that predicts the probability of a class. It works like linear regression, but the output is passed through a sigmoid function, which squashes any number into a value between 0 and 1.

Step by step: 1. Compute a score: $z = w \cdot x + b$ (same as linear regression). 2. Convert the score to a probability: $p = \sigma(z) = 1 / (1 + e^{-z})$. 3. Predict class 1 if $p \geq$ threshold (usually 0.5), otherwise class 0.

**Intuition:** the score z can be any real number. Positive z means "more likely class 1", negative means "more likely class 0", and z = 0 means "50-50".

### 11.2 Log-odds: what the model really learns

Logistic regression models the **log-odds** of the outcome as a linear function:

```
log( p / (1 − p) ) = w · x + b
```

- **Odds** = $p / (1 - p)$. If $p = 0.8$, odds = 4 (four-to-one in favour).
- **Log-odds** = log(odds). It can range from $-\infty$ to $+\infty$, which is why a linear model fits it.

**Interpreting a coefficient:** increasing feature $x_j$ by one unit multiplies the odds by $e^{w_j}$.

**Example:** A fault-prediction model has a coefficient of 0.69 for "vibration level" (per mm/s). $e^{0.69} \approx 2$. Each extra mm/s roughly doubles the odds of a fault, holding other features constant.

### 11.3 How it learns: maximum likelihood and log-loss

We want weights that make the observed labels most likely. This is equivalent to minimising the **log-loss** (binary cross-entropy):

```
Loss = −(1/n) Σ [ yᵢ log(pᵢ) + (1 − yᵢ) log(1 − pᵢ) ]
```

**Why not squared error?** With a sigmoid, squared error creates a non-convex problem with flat regions, which can trap the optimiser. Log-loss is convex for logistic regression, so gradient descent reliably finds the global minimum. Log-loss also punishes confident wrong predictions heavily.

**Example:** true label y = 1. - Predicted p = 0.9 → loss = −log(0.9) ≈ 0.11 (good) - Predicted p = 0.5 → loss ≈ 0.69 (unsure) - Predicted p = 0.05 → loss ≈ 3.0 (confidently wrong, heavily punished)

The gradient has a neat form: for each weight, it is the average of (predicted probability − actual label) times the feature value. This is why the algorithm trains quickly.

### 11.4 Decision threshold and class imbalance

The default threshold of 0.5 assumes equal costs for both errors and balanced classes. In practice: - Adjust the threshold using the precision-recall curve (Chapter 9). - Use **class_weight='balanced'** to make the model pay more attention to the rare class. - Remember that changing class weights changes the probabilities, not only the threshold. Be careful when interpreting calibrated probabilities after weighting.

### 11.5 Regularization

Same idea as linear regression: L2 (default in scikit-learn), L1, or Elastic Net. In scikit-learn, the parameter **C** is the inverse of regularization strength: small C means strong regularization. This trips up many candidates, so be ready to explain it.

### 11.6 Multi-class logistic regression

- **One-vs-rest:** train one binary model per class, pick the class with the highest probability.
- **Multinomial (softmax):** one model that outputs a probability for every class, summing to 1:

```
P(class k) = e^(z_k) / Σⱼ e^(z_j)
```

Softmax is also the output layer of most neural network classifiers.

### 11.7 Assumptions and limitations

- Assumes the log-odds are linear in the features. If the true boundary is curved, add polynomial or interaction features, or use a tree-based model.
- Sensitive to outliers and to multicollinearity.
- Needs feature scaling for regularization and for stable optimisation.
- Gives a linear decision boundary. Two classes that are separated by a circle cannot be split.

### 11.8 Interview questions for this chapter

**Q: Why is logistic regression called "regression" if it classifies?** It regresses the log-odds on the features with a linear model, then uses a threshold to classify. The regression happens on the log-odds scale.

**Q: What does the sigmoid do, and why not use a straight line?** A straight line can output values below 0 or above 1, which are not valid probabilities. The sigmoid maps any score into (0, 1) smoothly, and its shape matches how probabilities behave.

**Q: How do you interpret a coefficient?** A one-unit increase in the feature multiplies the odds by e^w. A positive coefficient raises the probability of class 1, a negative one lowers it. Standardise features first to compare coefficients across features.

**Q: Why do we use log-loss rather than accuracy to train?** Accuracy is not differentiable and changes only when a prediction crosses the threshold, so it gives no useful gradient. Log-loss is smooth and rewards well-calibrated probabilities.

**Q: How would you handle a rare class?** Use class weights or resampling, evaluate with precision, recall, PR-AUC instead of accuracy, and tune the threshold to the business cost.

**Q: Logistic regression vs decision tree, when do you choose which?** Logistic regression when relationships are roughly linear in the log-odds, data is limited, and interpretability matters. A tree when relationships are non-linear and interactions are strong, and when features are on very different scales or mixed types.

---

## Chapter 12: Decision Trees

### 12.1 The idea

A decision tree repeatedly asks questions about the features. Each question splits the data into two groups. At the end (the leaves), it predicts a class (the majority class) or a number (the average of the target).

**Analogy:** A doctor asks “Is the temperature above 38°C?” If yes, “Is there a cough?” If yes, “Recommend a test.” Each answer narrows the group. A tree does the same thing automatically.

Trees are popular because they are easy to explain, handle non-linear patterns and interactions naturally, and need little preprocessing (no scaling required).

### 12.2 How a split is chosen

At each node, the algorithm tries every feature and every possible threshold, and picks the split that makes the child groups **purest**.

**Classification impurity measures:** - **Gini impurity:** Gini $= 1 - \Sigma\, p_k^2$. Probability of mislabelling a random sample if you labelled it using the group’s class distribution. 0 means pure. - **Entropy:** $H = -\Sigma\, p_k \log_2 p_k$. Measures disorder in bits. 0 means pure. - **Information gain** = impurity before split − weighted impurity after split.

Gini is the default in scikit-learn. Gini and entropy usually give very similar trees. Gini is slightly faster to compute, since it has no logarithm.

**Regression trees** use variance reduction, or mean squared error, as the impurity. The leaf predicts the average target of its samples.

**Worked example (Gini):** A node has 10 samples: 6 faulty, 4 healthy. - Gini before $= 1 - (0.6^2 + 0.4^2) = 0.48$ - Split A produces: left = 4 faulty, 0 healthy (Gini 0); right = 2 faulty, 4 healthy (Gini $= 1 - (1/3)^2 - (2/3)^2 \approx 0.444$). - Weighted Gini after $= (4/10)(0) + (6/10)(0.444) \approx 0.267$ - Gain $= 0.48 - 0.267 = 0.213$. The tree compares this with every other candidate split.

### 12.3 Growing the tree and stopping

The algorithm keeps splitting until a stopping rule applies: - Maximum depth reached. - Node has fewer than a minimum number of samples. - Leaf would have fewer than a minimum number of samples. - No split improves impurity enough. - Node is pure.

Without limits, a tree grows until every leaf holds one training sample, which means it memorises the data. This is the main weakness of trees.

### 12.4 Pruning

Two families: - **Pre-pruning:** stop early, through the rules above (max depth, min samples per leaf). - **Post-pruning (cost-complexity pruning):** grow a large tree, then remove branches that do not improve performance enough, controlled by a complexity parameter alpha. Choose alpha by cross-validation.

**Key hyperparameters to name in an interview:** `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `ccp_alpha`.

### 12.5 Strengths and weaknesses

**Strengths** - Easy to visualise and explain to non-technical people. - Handles numeric and categorical data (after encoding), missing values (some implementations), and non-linear interactions. - No feature scaling needed. - Feature importance comes for free.

**Weaknesses** - **High variance:** a small change in the data can change the top split and the whole tree. - **Overfits** easily without limits. - Piecewise constant predictions, so it cannot extrapolate beyond the range of the training data (in regression). - Axis-aligned splits struggle with diagonal boundaries. A line at 45 degrees needs many splits to approximate. - Biased towards features with many distinct values when using impurity-based decisions.

### 12.6 Feature importance from a tree

Importance = total impurity reduction contributed by a feature across all its splits, weighted by how many samples reach those splits. It is quick, but can be misleading when features are correlated or have many levels. Permutation importance (shuffle a feature and measure the drop in score) is more reliable.

### 12.7 Interview questions for this chapter

**Q: How does a decision tree decide where to split?** It tries all features and thresholds, computes the impurity of the resulting groups (Gini or entropy for classification, variance for regression), and picks the split with the largest reduction in impurity.

**Q: Why does a deep tree overfit?** It keeps splitting until each leaf holds very few samples, so it learns the noise in those samples. It gets the training data almost perfectly right, but the rules it learned do not generalise.

**Q: How do you stop a tree from overfitting?** Limit depth and leaf size, prune with cross-validation, or use an ensemble such as random forest or gradient boosting, which reduce variance.

**Q: Do decision trees need feature scaling?** No. Splits depend on the order of values, not their magnitude. This is one of their practical advantages.

**Q: Can a tree extrapolate?** No. In regression, it predicts the average of training samples in the leaf, so it cannot predict values outside the training range. A linear model can extrapolate, a tree cannot.

**Q: Gini or entropy?** In practice, almost the same results. Gini is default and faster. Entropy is sometimes preferred when a more information-theoretic view is wanted.

---

## Chapter 13: Random Forests and Ensemble Methods

### 13.1 The idea: wisdom of crowds

One decision tree is unstable. A random forest builds many trees and combines their predictions. Individually each tree is noisy, but their errors partly cancel out when averaged or voted.

**Analogy:** Ask 500 people to guess the number of beans in a jar. Each guess is poor, but the average is often very close. The crowd works because individual errors are somewhat independent.

For this to work, the trees must be **diverse**. Random forest creates diversity in two ways.

### 13.2 How it works

1. **Bootstrap sampling (bagging):** for each tree, draw n rows with replacement from the n training rows. About 63.2% of unique rows appear in each bootstrap sample. The remaining ~36.8% are “out-of-bag” for that tree.
2. **Random feature subset:** at each split, the tree considers only a random subset of features, typically √p for classification and p/3 for regression (scikit-learn defaults have changed over versions, so say “a random subset, tuned with `max_features`”).
3. **Grow each tree deep** (usually without pruning). Individual trees have low bias and high variance, but averaging reduces the variance.
4. **Aggregate:** majority vote for classification, average for regression. Scikit-learn can also return averaged class probabilities.

### 13.3 Why random feature selection matters

Suppose one feature is very strong. Without feature randomness, every tree would split on it first, and the trees would look alike. Their errors would be correlated, and averaging would help much less. Random feature subsets decorrelate the trees.

### 13.4 Out-of-bag (OOB) error

For each training row, predict using only the trees that did not see that row during training. Average those predictions and compare with the true label. This gives a validation estimate without a separate hold-out set. It is useful for quick checks, but for final reporting, still use a proper test set or cross-validation.

### 13.5 Key hyperparameters

- **n_estimators:** number of trees. More trees usually help, with diminishing returns, and never increase overfitting in the usual sense. The cost is time and memory.
- **max_features:** features considered per split. Smaller values give more diversity and lower correlation between trees.
- **max_depth, min_samples_leaf:** control tree size. Forests usually use deeper trees than single trees, but limits still help on noisy data.
- **bootstrap:** whether to use bootstrap samples.
- **class_weight:** handles imbalance.

### 13.6 Feature importance in forests

- **Impurity-based (Mean Decrease in Impurity):** fast, built-in. Biased towards high-cardinality numeric features and can be unreliable when features are correlated.

- **Permutation importance:** shuffle one feature on held-out data and measure the drop in score. Model-agnostic, more reliable, slower.
- **SHAP values:** explain each prediction and aggregate. Good for explaining to business stakeholders.

Always say which one you use and why.

### 13.7 Strengths and weaknesses

**Strengths** - Strong performance out of the box, with little tuning. - Handles non-linear patterns and interactions. - Robust to outliers and irrelevant features (mostly). - No scaling needed. - Gives OOB estimates and feature importance. - Can be parallelised easily, since trees are independent.

**Weaknesses** - Large models: many trees, deep trees, large memory. - Slower predictions than a single tree or linear model. - Less interpretable than a single tree or logistic regression. - Cannot extrapolate in regression. - Probabilities may be poorly calibrated. - Performance can plateau below gradient boosting on structured data.

### 13.8 Boosting: the other ensemble family

Bagging builds trees in parallel and averages them to reduce variance. **Boosting** builds trees in sequence, where each new tree focuses on the mistakes of the previous ones, which reduces bias.

- **AdaBoost:** re-weights misclassified samples so the next learner focuses on them.
- **Gradient boosting:** each new tree fits the negative gradient of the loss (the residual errors, in regression). The final model is a weighted sum of trees.
- **XGBoost, LightGBM, CatBoost:** efficient, regularised implementations. They dominate many tabular-data competitions and industry projects.

Key boosting hyperparameters: learning rate (shrinkage), number of trees, tree depth, subsampling, and regularization terms. Lower learning rate needs more trees.

**Bagging vs boosting in one line:** bagging reduces variance by averaging independent trees; boosting reduces bias by fitting trees one after another to the errors.

### 13.9 Interview questions for this chapter

**Q: Why does random forest beat a single decision tree?** A single tree has high variance. Averaging many trees trained on different bootstrap samples with random feature subsets reduces the variance while keeping the low bias of deep trees.

**Q: What is bootstrap sampling and why sample with replacement?** It creates new training sets of the same size by sampling rows with replacement. Each set is slightly different, which creates diversity among the trees.

**Q: What is out-of-bag error?** Each row is left out of about 37% of the trees. Predicting each row with only the trees that didn’t train on it gives an honest error estimate for free.

**Q: Why not just use more and deeper trees to get better results?** More trees help until the error stops improving. Very deep trees keep low bias but the forest still has limited capacity to reduce variance further. Too many trees just costs compute. Tune using cross-validation.

**Q: Random forest or gradient boosting?** Random forest is easier to tune and more robust with default settings. Gradient boosting usually achieves higher accuracy on structured data but needs careful tuning and is more prone to overfitting if neglected. I’d start with a random forest as a baseline, then try gradient boosting.

**Q: Your forest says feature A is the most important. Can you trust that?** Not without checking. Impurity-based importance can favour features with many unique values, and correlated features split importance between them. I’d confirm with permutation importance on a validation set, and check whether the feature is a leak.

---

## Chapter 14: Naive Bayes

### 14.1 The idea

Naive Bayes uses Bayes’ theorem (Chapter 2) to predict a class given features. It makes a strong simplifying assumption: given the class, all features are independent of each other. That assumption is almost never true, which is why the method is called “naive”. But it often works surprisingly well, especially for text, and it is very fast.

**Analogy:** A doctor diagnosing flu looks at fever, cough and body aches separately, multiplies how common each symptom is among flu patients, and compares with the same calculation for non-flu patients. The doctor ignores that fever and body aches often come together. Naive Bayes does the same.

### 14.2 The formula

$$P(\text{class} \mid x_1, \ldots, x_p) \propto P(\text{class}) \times \prod P(x_i \mid \text{class})$$

The prediction is the class with the highest value. The denominator (the evidence) is the same for every class, so it can be ignored when comparing.

### 14.3 Three common variants

- **Gaussian Naive Bayes:** continuous features, assumes each feature is normal within each class. Estimate the mean and variance per class and feature. Use for sensor readings.
- **Multinomial Naive Bayes:** count features, such as word counts in documents. Standard for text classification.
- **Bernoulli Naive Bayes:** binary features, such as whether a word is present or absent. Good for short texts.

### 14.4 Worked example: spam filter, two words

Prior: P(spam) = 0.4, P(ham) = 0.6.

Likelihoods: - P(“free” | spam) = 0.30, P(“free” | ham) = 0.05 - P(“meeting” | spam) = 0.02, P(“meeting” | ham) = 0.20

Email contains “free” and “meeting”.

Unnormalised scores: - Spam: 0.4 × 0.30 × 0.02 = 0.0024 - Ham: 0.6 × 0.05 × 0.20 = 0.0060

Normalise: P(spam) = 0.0024 / (0.0024 + 0.0060) ≈ 0.286. So the email is classified as ham, even though “free” points to spam. The word “meeting” is strong evidence for ham and outweighs it.

### 14.5 Laplace smoothing

If a word never appeared in spam during training, P(word | spam) = 0, and the whole product becomes zero, regardless of other words. Laplace (add-one) smoothing fixes this:

$$P(\text{word} \mid \text{class}) = \frac{(\text{count of word in class} + \alpha)}{(\text{total words in class} + \alpha \times \text{vocabulary size})}$$

With α = 1 this is add-one smoothing. Smoothing also handles words not seen at all during training, as long as the vocabulary includes them.

### 14.6 Working in log space

Multiplying many small probabilities underflows to zero on a computer. The fix is to add logarithms instead of multiplying:

$$\log P(\text{class} \mid x) \propto \log P(\text{class}) + \Sigma \log P(x_i \mid \text{class})$$

Logs turn products into sums, and the ranking of classes stays the same. Scikit-learn does this internally.

### 14.7 Why it works despite the wrong assumption

- For classification, you only need the **ranking** of class probabilities to be right, not their exact values. Independence errors often push all classes in similar ways, so the winner can still be correct.
- It needs little data to estimate probabilities, which helps with small samples.
- High-dimensional sparse data (text) suits it well.

### 14.8 Strengths and weaknesses

**Strengths:** very fast to train and predict, works with small data, handles many features, good baseline for text, simple to explain, handles streaming updates easily.

**Weaknesses:** the independence assumption fails with correlated features, and then probabilities are poorly calibrated. Weaker than logistic regression or boosting when enough data is available and features interact. Zero-frequency issue needs smoothing. Gaussian version assumes normal features.

### 14.9 Interview questions for this chapter

**Q: Why is it called naive?** Because it assumes features are conditionally independent given the class. Real features are often correlated, so the assumption is naive, but the classifier often still ranks classes well.

**Q: When would you use Naive Bayes?** As a fast baseline, for text classification (spam, sentiment, topic), with small datasets, or when prediction speed and simplicity matter more than top accuracy.

**Q: What is Laplace smoothing and why is it needed?** It adds a small count to every feature-class combination so that no probability is exactly zero. Without it, a single unseen word would make the whole class probability zero.

**Q: Why do we use logarithms?** To avoid numerical underflow from multiplying many small probabilities, and to turn products into sums. The predicted class does not change.

**Q: Naive Bayes vs logistic regression for text?** Naive Bayes trains faster and works with less data. Logistic regression usually gives better accuracy with enough data, and gives better-calibrated probabilities after training. I'd try both, with TF-IDF features for the linear model.

---

## Chapter 15: k-Nearest Neighbours (k-NN)

### 15.1 The idea

k-NN is the simplest machine learning method. To predict a new point, find the k training points closest to it and combine their labels (classification, by majority vote) or values (regression, by average).

It is an **instance-based** or **lazy** learner: it stores the training data and does the work at prediction time. There is no real training step beyond storing the data and perhaps building a search index.

**Analogy:** To guess what kind of music someone likes, look at the five friends who are most similar to them and see what those friends listen to.

### 15.2 The algorithm

1. Choose k and a distance metric.
2. Compute the distance from the query point to every training point.
3. Select the k smallest distances.
4. Classification: majority vote among the k labels. Regression: mean (or distance-weighted mean) of their values.

### 15.3 Distance metrics

- **Euclidean:** straight-line distance, sqrt(Σ $(x_i - y_i)^2$). Default.
- **Manhattan:** sum of absolute differences. More robust to outliers in one feature.
- **Minkowski:** general form with parameter p (p = 1 Manhattan, p = 2 Euclidean).
- **Cosine distance:** 1 − cosine similarity. Ignores vector length, good for text and embeddings.
- **Hamming:** number of differing positions, for categorical or binary features.

**Example:** points A = (1, 2) and B = (4, 6). Euclidean = √(9 + 16) = 5. Manhattan = 3 + 4 = 7.

### 15.4 Choosing k

- **Small k (1 or 3):** flexible boundary, sensitive to noise, high variance.
- **Large k:** smoother boundary, more stable, can smooth over small but real patterns, high bias.
- Use odd k for two-class problems to avoid ties.
- Choose k by cross-validation. Plot validation error against k and look for the value where it stops improving.

**Weighted k-NN:** closer neighbours get more weight, for example 1/distance. Reduces the influence of distant points within the k.

### 15.5 Feature scaling is essential

Distance depends on units. If income is in rupees (values in lakhs) and age in years, income dominates the distance completely. Standardise (or min-max scale) all numeric features before k-NN. Fit the scaler on training data only.

Also consider feature weighting or removing irrelevant features, because every irrelevant feature adds noise to every distance.

### 15.6 Speed and memory

- **Brute force:** compute all distances, O(n × d) per query. Fine for small datasets.
- **KD-tree and Ball tree:** spatial index structures that speed up search in low to moderate dimensions (roughly up to 20).
- **Approximate nearest neighbours** (for example, HNSW, FAISS): fast search in high dimensions, trading a small amount of accuracy for large speed-ups. Used in semantic search and recommendation.

Memory grows with the training set, since everything is stored.

### 15.7 Curse of dimensionality

In high dimensions: - Distances between random points become similar. The gap between the nearest and farthest point shrinks, so “nearest” carries less meaning. - The volume needed to contain the k neighbours grows fast, so neighbours are no longer near.

Remedies: reduce dimensions (PCA, feature selection), use domain knowledge to build meaningful features, or choose a different model.

### 15.8 Interview questions for this chapter

**Q: Why is k-NN called a lazy learner?** It does not build a model during training. It stores the data and computes at prediction time.

**Q: Why must you scale features for k-NN?** Because distance is computed across all features. Unscaled features with large ranges would dominate the distance, so the model would effectively ignore the others.

**Q: How do you choose k?** Cross-validation. Small k gives a flexible but noisy boundary. Large k gives a smoother but possibly biased one. I’d look for the k with the lowest validation error, using an odd number for binary problems.

**Q: What happens to k-NN in high dimensions?** Distances concentrate, so nearest and farthest neighbours look alike, and the method degrades. I’d reduce dimensions first or pick a different model.

**Q: When is k-NN a good choice?** Small to medium datasets, low-dimensional data, non-linear boundaries, and when interpretability by example matters (“this machine looks like these three past failures”). It is also used as a baseline and in recommendation systems.

**Q: How would you make k-NN faster on a big dataset?** Use a KD-tree or ball tree for low dimensions, or approximate nearest neighbour search for high dimensions. Reduce features, and sample or cluster the training set.

---

## Chapter 16: k-Means and Clustering

### 16.1 The idea

Clustering groups similar items together without labels. k-means is the most common clustering algorithm. You choose k, and it finds k centres (centroids) such that each point belongs to the cluster with the closest centre.

**Analogy:** You are placing k coffee shops in a city so that each neighbourhood is as close as possible to one of them. You move the shops around until each one sits at the centre of its neighbourhood.

### 16.2 The algorithm (Lloyd’s algorithm)

1. Choose k and initial centroids.
2. **Assignment step:** assign each point to its nearest centroid.
3. **Update step:** move each centroid to the mean of the points assigned to it.
4. Repeat steps 2 and 3 until assignments stop changing (or centroids move very little, or a maximum number of iterations is reached).

Each step reduces the objective, so the algorithm converges, but only to a **local** minimum.

**Objective (inertia or within-cluster sum of squares):**

```
J = Σ over clusters Σ over points in cluster ||x − μ_cluster||²
```

### 16.3 Worked example

Points: 1, 2, 8, 9. k = 2. Start centroids at 1 and 8. - Assignment: 1 and 2 go to centroid 1 (distances 0 and 1). 8 and 9 go to centroid 8 (distances 0 and 1). - Update: new centroids are (1 + 2)/2 = 1.5 and (8 + 9)/2 = 8.5. - Next assignment is unchanged, so the algorithm stops.

### 16.4 Initialisation matters

Random initial centroids can lead to poor local minima. Solutions: - **k-means++:** choose the first centroid at random, then choose each next centroid with probability proportional to its squared distance from the nearest existing centroid. This spreads the initial centroids out. It is the default in scikit-learn. - **Multiple restarts (n_init):** run several times with different starts and keep the best inertia.

### 16.5 Choosing k

There is no single correct k. Use several signals together:

- **Elbow method:** plot inertia against k. Look for the point where the curve bends and extra clusters give little improvement. Judgement is needed, because the bend is often not sharp.
- **Silhouette score:** for each point, a = average distance to points in its own cluster, b = average distance to points in the nearest other cluster. Silhouette = (b − a) / max(a, b), ranging from −1 to +1. Higher is better. The average over all points gives a score for k.
- **Davies-Bouldin index:** ratio of within-cluster scatter to between-cluster separation. Lower is better.
- **Business sense:** can you describe each cluster and act on it? Clusters that are statistically neat but meaningless are not useful.

### 16.6 Assumptions and limitations

- You must choose k in advance.
- Assumes clusters are roughly spherical, similar in size and density.
- Sensitive to outliers, which pull centroids.
- Sensitive to feature scaling. Standardise features first.
- Works with Euclidean distance on numeric features. For categorical data, use k-modes, or encode carefully.
- Hard assignment: each point belongs to exactly one cluster.

### 16.7 Other clustering methods (know the names and when to use them)

- **Hierarchical clustering:** builds a tree (dendrogram) by merging the closest clusters step by step (agglomerative), or by splitting. No need to pick k in advance, but slow on large data. Good for small data and exploring structure.
- **DBSCAN:** groups dense regions and marks sparse points as noise. Finds arbitrary shapes and detects outliers. Needs eps and min_samples, which are hard to tune.
- **Gaussian Mixture Models (GMM):** soft clustering. Each cluster is a Gaussian, and each point gets a probability of belonging to each cluster. Handles elliptical clusters.
- **HDBSCAN:** a more robust, density-based variant that picks the number of clusters automatically.

### 16.8 Applications in industry

- Grouping machines by operating profile, to compare like with like.
- Customer segmentation for marketing or credit offers.
- Grouping production batches with similar sensor signatures.
- Anomaly detection: points far from every centroid are unusual.

### 16.9 Interview questions for this chapter

**Q: Explain k-means in simple words.** It places k centres, assigns each point to its nearest centre, moves each centre to the average of its points, and repeats until nothing changes.

**Q: Is k-means supervised or unsupervised? Why does that matter?** Unsupervised: there are no labels. So evaluation is harder. You rely on internal measures like silhouette, plus business interpretation and stability checks.

**Q: How do you choose k?** I combine the elbow method, silhouette score, and whether the clusters are interpretable and useful. I also check stability: do the clusters stay similar across different random seeds or samples?

**Q: Why do we use k-means++?** Random starts can place two centroids in the same dense area, which gives poor clusters. k-means++ spreads the starting points, which leads to better and more consistent results.

**Q: What if clusters have different sizes or shapes?** k-means assumes roughly spherical, similar-sized clusters, so it may split large clusters or merge small ones. Try GMM for elliptical clusters, or DBSCAN and HDBSCAN for irregular shapes.

**Q: Do you need to scale before k-means?** Yes. Distances drive the algorithm, so features with large ranges dominate. Standardise numeric features, and be careful with categorical features.

**Q: How would you use clustering to detect anomalies?** Fit clusters on normal operating data. For a new point, measure its distance to the nearest centroid. Large distances, above a threshold chosen from historical data, flag anomalies.

---

## Chapter 17: Principal Component Analysis (PCA)

### 17.1 The idea

PCA is a method for reducing the number of features while keeping as much information as possible. It finds new axes (principal components) that point in the directions of greatest variance in the data. Each component is a combination of the original features.

**Analogy:** Think of a 3D cloud of points shaped like a flat, tilted pancake. Most of the spread is across the pancake's surface. You can describe the cloud almost as well with a 2D picture of the surface, and lose only the thin thickness.

### 17.2 What the components are

- **PC1:** the direction along which the data varies most.
- **PC2:** the direction of greatest remaining variance, perpendicular to PC1.
- And so on. All components are perpendicular (orthogonal) to each other, so they are uncorrelated with one another.

Each component has an **explained variance**, and the components are sorted from largest to smallest.

### 17.3 Steps

1. Centre the data (subtract each feature's mean). Usually also standardise each feature to unit variance, unless the features share the same units and that scale is meaningful.
2. Compute the covariance matrix of the features (or use SVD directly on the data matrix, which is more numerically stable and what libraries do).
3. Find the eigenvectors (components) and eigenvalues (variance along each component).
4. Sort by eigenvalue, largest first.
5. Keep the first k components and project the data onto them: $Z = X \cdot W_k$.

**SVD view:** $X = U \Sigma V^T$. The columns of V are the principal directions, and the squared singular values divided by $(n - 1)$ give the variances. Scikit-learn's PCA uses SVD.

### 17.4 Choosing the number of components

- **Explained variance ratio:** share of total variance captured by the first k components. A common rule is to keep enough components for 90–95% of variance.
- **Scree plot:** plot eigenvalues in order and look for the "elbow".
- **Downstream performance:** choose k that gives the best cross-validated model score.

**Example:** eigenvalues 4.0, 2.5, 1.0, 0.3, 0.2 (total 8.0). Cumulative variance: 50%, 81%, 93%, 97%, 100%. Keeping 3 components gives 93% of the variance.

### 17.5 Uses

- Reducing many correlated sensor channels into a few components.
- Removing multicollinearity before regression.
- Speeding up algorithms like k-NN or clustering.
- Visualising high-dimensional data in 2D or 3D.
- Denoising: small-variance components are often noise.
- Compressing images (principal components of image patches).

### 17.6 Limitations

- **Linear:** finds linear combinations only. Curved structure is missed. Kernel PCA handles non-linear structure.
- **Hard to interpret:** each component mixes many features. You can look at the loadings (weights) to see which original features contribute most.

- **Ignores the target:** PCA maximises variance, not prediction. The direction with the most variance may have nothing to do with the label. For supervised tasks, consider PLS or use the features directly with regularization.
- **Sensitive to scaling:** large-unit features dominate if not standardised.
- **Sensitive to outliers:** outliers can create false directions of variance.

### 17.7 PCA vs t-SNE and UMAP

- **PCA:** linear, preserves global structure and distances roughly, fast, deterministic. Good for preprocessing and modelling.
- **t-SNE and UMAP:** non-linear, mainly for visualisation. Preserve local neighbourhoods, not global distances. Distances between clusters in these plots are not meaningful, and results vary with settings. Do not use them as features for modelling without care.

### 17.8 Interview questions for this chapter

**Q: What does PCA actually do?** It rotates the data onto new axes ordered by variance, then keeps the first few axes. It compresses the data while keeping the directions with the most information.

**Q: Why do we standardise before PCA?** PCA is driven by variance. A feature measured in large units has large variance just because of its units, so it would dominate the components. Standardising puts features on equal footing.

**Q: Is PCA supervised or unsupervised?** Unsupervised. It ignores the target. This is a limitation if the most informative direction for prediction has low variance.

**Q: How do you decide how many components to keep?** I look at the cumulative explained variance (for example, keep 90–95%) and the scree plot. Then I check whether a model trained on the reduced data performs as well as on all features.

**Q: PCA vs feature selection?** PCA creates new combined features. Feature selection keeps a subset of the original features. Selection keeps interpretability; PCA can capture more variance per dimension but the components are harder to explain.

**Q: What are the risks of using PCA before a model?** Losing a low-variance direction that actually matters for prediction, creating features that are hard to explain to stakeholders, and fitting PCA on the full data (leakage). Fit PCA on training data only, inside the pipeline.

---

## Chapter 18: Neural Networks and Deep Learning

### 18.1 The idea

A neural network is a chain of simple computing units (neurons) arranged in layers. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function. Stacking many layers lets the network learn very complex patterns, such as recognising a face or a defect in an image.

**Why non-linearity is essential:** if every layer were just a weighted sum, the whole network would collapse into one linear function. Activation functions break this and let the network model curved relationships.

### 18.2 The perceptron (the simplest neural network)

A single neuron with a step activation: output 1 if the weighted sum exceeds a threshold, else 0. It can learn AND or OR, but cannot learn XOR, because XOR is not linearly separable. This limitation led to multi-layer networks, which can learn XOR by combining two linear boundaries.

### 18.3 Multi-layer perceptron (MLP)

Structure: - **Input layer:** the features. - **Hidden layers:** learned representations. Each neuron combines the previous layer's outputs. - **Output layer:** the prediction. Sigmoid for binary, softmax for multi-class, linear for regression.

For a layer with inputs a, weights W, bias b:

```
z = W·a + b
a_next = activation(z)
```

**Example neuron:** inputs $x_1 = 1$, $x_2 = 2$; weights 0.5 and $-0.3$; bias 0.1. $z = 0.5 \times 1 + (-0.3) \times 2 + 0.1 = 0.0$. With ReLU, output = 0. With sigmoid, output = 0.5.

### 18.4 Activation functions

- **Sigmoid:** 1 / (1 + e^(−z)). Output in (0, 1). Saturates for large |z|, so gradients vanish. Mainly used for binary output.
- **Tanh:** output in (−1, 1), zero-centred. Also saturates.
- **ReLU:** max(0, z). Fast, does not saturate for positive values, and sparse. Default for hidden layers. Its problem is "dying ReLU", where neurons get stuck at zero.
- **Leaky ReLU, ELU, GELU:** variants that keep a small gradient for negative values. GELU is common in transformer models.
- **Softmax:** converts a vector of scores into probabilities that sum to 1. Output layer for multi-class.

### 18.5 Loss functions

- **Regression:** mean squared error (MSE) or mean absolute error (MAE). Huber loss combines both, and is robust to outliers.
- **Binary classification:** binary cross-entropy (log-loss) with a sigmoid output.
- **Multi-class classification:** categorical cross-entropy with softmax output.

Choose the loss that matches the output layer. Using MSE for classification trains poorly, because the gradients are weak for probabilities.

### 18.6 Gradient descent and its variants

The network learns by adjusting weights to reduce the loss.

- **Batch gradient descent:** use all data for each update. Accurate but slow.
- **Stochastic gradient descent (SGD):** use one sample per update. Fast but noisy.
- **Mini-batch gradient descent:** use a small batch (for example, 32 to 256 samples). The standard in practice, because it balances speed and stability and uses hardware efficiently.

**Terms:** - **Batch size:** samples per update. - **Iteration:** one update. - **Epoch:** one full pass through the training data.

**Learning rate:** the step size. Too high: loss oscillates or diverges. Too low: very slow. Learning rate schedules (decay, warm-up, cosine) help.

**Momentum:** keeps a running average of past gradients, so updates build speed in consistent directions and dampen oscillations.

**Adam:** combines momentum with per-parameter adaptive learning rates. Default choice for most deep learning. Typical settings: learning rate 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

### 18.7 Backpropagation (the idea in plain words)

Backpropagation computes how much each weight contributed to the error. 1. Run a forward pass: compute the prediction and loss. 2. Compute the gradient of the loss with respect to the output. 3. Use the chain rule to pass the gradient backward layer by layer, computing how each weight affects the loss. 4. Update the weights using the optimiser.

You do not need to derive this on a whiteboard. Be ready to say: "The error flows backward through the network, and each weight receives its share of the blame, computed with the chain rule."

### 18.8 Weight initialisation

Starting weights must not be too large or too small, or signals and gradients shrink or explode through layers. - **Xavier (Glorot) initialisation:** suited to sigmoid and tanh. - **He initialisation:** suited to ReLU. - Zero initialisation is a bad idea: all neurons in a layer learn the same thing.

### 18.9 Regularization and generalisation

- **L2 weight decay:** penalises large weights. In AdamW, decoupled from the gradient update, which works better.
- **Dropout:** randomly sets a fraction of activations to zero during training (typically 0.2 to 0.5). Forces the network to learn redundant, robust features. Turned off at prediction time.
- **Early stopping:** monitor validation loss, stop when it has not improved for a set number of epochs (patience), and restore the best weights.
- **Data augmentation:** create new training examples by transforming existing ones (flips, rotations, noise, crops for images; jitter for signals). Very effective when data is limited.
- **Batch normalisation:** normalises activations within each mini-batch, which stabilises and speeds up training. Also has a mild regularising effect.

### 18.10 Common training problems and fixes

- **Vanishing gradients:** gradients shrink to near zero in deep networks with sigmoid or tanh. Fixes: ReLU family, good initialisation, batch normalisation, residual connections.
- **Exploding gradients:** gradients grow very large. Fixes: gradient clipping, lower learning rate, normalisation.
- **Dead ReLUs:** many neurons output zero permanently. Fixes: Leaky ReLU, lower learning rate.
- **Overfitting:** training loss keeps falling while validation loss rises. Fixes: regularization, augmentation, smaller network, more data.
- **Underfitting:** both losses high. Fixes: bigger network, longer training, better features.
- **Loss not decreasing:** check the learning rate, data pipeline (labels and inputs aligned), and that the loss matches the output activation.

### 18.11 Convolutional neural networks (CNNs)

CNNs are built for grid data such as images, and also work for 1D signals and 3D medical volumes.

- **Convolution:** a small filter (kernel), for example 3×3, slides over the image. At each position, it multiplies and sums the overlapping values, producing a feature map. The same filter is reused across the image, so the model has far fewer parameters than a fully connected network.
- **Filters learn features:** early layers detect edges and textures, middle layers detect parts, later layers detect objects.
- **Pooling:** downsamples the feature maps (max pooling takes the maximum in each window). Gives some tolerance to small shifts.
- **Stride and padding:** control how far the filter moves and whether the image edges are preserved.
- **Architecture:** conv layers, then pooling, then dense layers. Well-known examples are LeNet, AlexNet, VGG, ResNet, and U-Net (for segmentation, widely used in medical imaging).

**Transfer learning:** start from a network pretrained on a large dataset (like ImageNet), replace the final layer, and fine-tune on your data. Very useful when labelled data is limited, which is common in medical and industrial imaging.

### 18.12 Neural networks vs classical ML: when to choose which

Neural networks are strong when: - Data is unstructured (images, audio, text, signals) or very large. - Patterns are complex and features are hard to design by hand. - You can afford training time and compute.

Classical models (linear models, trees, boosting) are often better when: - Data is tabular with modest size. - Interpretability is required. - You need quick iteration and a simpler deployment.

**Interview tip:** Gradient boosting often beats neural networks on tabular data. Say this to show judgement, not hype.

### 18.13 Interview questions for this chapter

**Q: What is a neural network, in simple words?** A stack of layers of simple units. Each unit weights its inputs, adds a bias, and applies a non-linear function. Training adjusts the weights so the outputs match the labels.

**Q: Why do we need activation functions?** Without non-linear activations, stacked layers reduce to a single linear transformation, so the network could only learn linear patterns.

**Q: Why ReLU instead of sigmoid in hidden layers?** ReLU does not saturate for positive inputs, so gradients do not vanish as quickly. It is also cheap to compute. Sigmoid saturates, so deep networks with sigmoid can learn very slowly.

**Q: What is backpropagation?** An efficient way to compute the gradient of the loss with respect to every weight, using the chain rule, by passing errors backward from the output through the layers.

**Q: What is the role of the learning rate?** It sets the size of each update. Too large makes training unstable. Too small makes training very slow or stuck. I'd tune it, often with a schedule, and watch the loss curve.

**Q: How do you prevent overfitting in a neural network?** Use more data or augmentation, dropout, weight decay, early stopping, a smaller architecture, and batch normalisation. I'd monitor the gap between training and validation loss.

**Q: Explain Adam simply.** Adam is gradient descent that keeps a running average of past gradients (momentum) and adapts the step size for each weight based on how large its gradients have been. It usually trains faster and needs less tuning than plain SGD.

**Q: What is a CNN and why is it good for images?** A network that uses small filters slid across the image, so it detects local patterns anywhere in the image using shared weights. It needs far fewer parameters than a fully connected network and builds up from edges to objects.

**Q: What is transfer learning and when do you use it?** Reusing a model pretrained on a large dataset and fine-tuning it on a smaller task. Use it when labelled data is limited, which is common in medical imaging.

**Q: Would you use a neural network for a tabular sensor dataset of 20,000 rows?** Probably not first. I'd start with gradient boosting or a random forest, which are usually strong and easier to tune and explain. A neural network would be worth trying only if classical models plateau and there is enough data.

---

## Chapter 19: Feature Engineering, Preprocessing, and Pipelines

### 19.1 The idea

Models only see the features you give them. Feature engineering means turning raw data into inputs that make patterns easier to learn. In many real projects, this matters more than picking the algorithm.

**Analogy:** For predicting whether a machine will fail, raw timestamps are not useful. But "hours since last maintenance" and "number of high-temperature events in the last 24 hours" are very informative. Good features encode domain knowledge.

### 19.2 Handling missing values

First, ask **why** the data is missing. Missing at random is different from missing because something happened (for example, a sensor drops out during a fault).

Options: - **Drop rows:** only if very few rows are affected. - **Drop columns:** if most values are missing. - **Simple imputation:** mean or median for numeric, most frequent for categorical, or a constant such as "unknown". - **Indicator column:** add a flag "was missing" so the model can learn from missingness itself. Often very useful. - **Model-based imputation:** KNN imputer or iterative imputer, which predicts missing values from other features.

**Important:** fit the imputer on training data only, then apply it to validation and test data.

### 19.3 Scaling numeric features

- **Standardisation:** z = (x − mean) / std. Mean 0, std 1. Needed for distance-based methods, linear models with regularization, PCA, and neural networks.
- **Min-max scaling:** (x − min) / (max − min). Range [0, 1]. Sensitive to outliers.
- **Robust scaling:** (x − median) / IQR. Resistant to outliers.
- **Log transform:** for right-skewed data like prices or counts. Use log(1 + x) if zeros exist.

Tree-based models do not need scaling.

### 19.4 Encoding categorical features

- **One-hot encoding:** one binary column per category. Good for nominal categories with few levels. Use drop='first' in linear models to avoid redundancy.
- **Ordinal encoding:** integers for ordered categories (low, medium, high). Only when the order is real.
- **Target encoding (mean encoding):** replace each category with the average target value for that category. Powerful for high-cardinality features like city or machine ID, but very prone to leakage. Compute it using cross-fitting: encode the training fold using out-of-fold means, and apply the training statistics to validation or test data. Add smoothing toward the global mean for rare categories.
- **Frequency encoding:** replace category with how often it appears. Simple and sometimes effective.
- **Embeddings:** learned dense vectors for categories, used in neural networks.

**Pitfall:** label encoding a nominal feature (city = 1, 2, 3) implies a false order to linear models. Trees handle it better, but one-hot is safer.

### 19.5 Creating new features

Common patterns: - **Date and time:** hour, day of week, month, is_weekend, holiday flag, time since an event. - **Ratios and differences:** price per unit, change from the previous reading. - **Aggregations:** mean, max, min, standard deviation, count over a time window or group (for example, average vibration over the last 60 minutes). - **Lags and rolling statistics** for time series: value 1 hour ago, rolling 24-hour mean. - **Interaction terms:** product of two features, so a linear model can capture combined effects. - **Polynomial features:** squares and cross-terms for curved relationships in linear models. - **Domain features:** for machine data, frequency-domain features from FFT, kurtosis of vibration, or RMS value.

**Warning:** every feature must be computable at prediction time. Aggregations over a window must not use future data.

### 19.6 Feature selection

Reducing features can improve generalisation and interpretability. - **Filter methods:** rank features by statistics like correlation with the target, chi-square, or mutual information. Fast, model-independent, ignores interactions. - **Wrapper methods:** search subsets using a model (recursive feature elimination). Accurate, expensive. - **Embedded methods:** selection during training, such as L1 regularization (Lasso) or tree-based importance. - **Remove:** near-constant features, duplicates, highly correlated pairs (keep one), and features that leak the target.

### 19.7 Handling imbalanced data

- **Class weights:** tell the loss function to penalise errors on the rare class more. Simple and effective.
- **Random undersampling:** remove majority-class rows. Loses information.
- **Random oversampling:** duplicate minority rows. Risks overfitting.
- **SMOTE:** creates synthetic minority samples by interpolating between a minority point and one of its nearest minority neighbours. Apply only to the training fold, never to validation or test data.
- **Threshold tuning** and better metrics (Chapter 9).
- **Anomaly detection framing:** if the positive class is extremely rare, sometimes treat it as anomaly detection.

### 19.8 Data leakage revisited: the pipeline rule

Every data-dependent step (imputation, scaling, encoding, feature selection, PCA, SMOTE, target encoding) must be fitted on the training data only, inside each cross-validation fold. Scikit-learn's **Pipeline** and **ColumnTransformer** enforce this:

```
        from sklearn.pipeline import Pipeline
        from sklearn.compose import ColumnTransformer
        from sklearn.preprocessing import StandardScaler, OneHotEncoder
        from sklearn.impute import SimpleImputer
        from sklearn.ensemble import RandomForestClassifier

        numeric = Pipeline([("impute", SimpleImputer(strategy="median")),
                            ("scale", StandardScaler())])
        categorical = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                                ("onehot", OneHotEncoder(handle_unknown="ignore"))])

        prep = ColumnTransformer([("num", numeric, num_cols),
                                  ("cat", categorical, cat_cols)])

        model = Pipeline([("prep", prep),
                          ("clf", RandomForestClassifier(n_estimators=300, random_state=42))])

        model.fit(X_train, y_train)          # all fitting happens on training data only
        model.predict(X_test)
```

Benefits: no leakage in cross-validation, one object to deploy, and reproducibility.

### 19.9 Interview questions for this chapter

**Q: How do you handle missing values?** First I find out why values are missing. Then I choose between dropping, simple imputation with a missing-indicator column, or model-based imputation. I fit the imputer on training data only.

**Q: Why do we need a missing-value indicator?** Missingness can carry information. For example, a sensor that goes silent may signal a fault. The indicator lets the model use that signal.

**Q: What is target encoding and what is its danger?** Replacing a category with the average target for that category. The danger is leakage: if you compute it on the same data the model learns from, the feature encodes the answer. Use cross-fitting and smoothing.

**Q: One-hot or label encoding?** One-hot for nominal categories, because label encoding implies a false order that linear models would exploit. Label encoding only for ordered categories, or for tree models where it is less harmful.

**Q: How would you handle a dataset where 1% of rows are positive?** Use class weights or SMOTE on the training folds only, evaluate with precision, recall and PR-AUC, and tune the decision threshold to the business cost. I'd also check that the positives are not duplicates or leaked signals.

**Q: What is a pipeline and why use one?** A pipeline chains preprocessing and modelling so that fitting and prediction apply the same steps. It prevents leakage during cross-validation and makes deployment consistent.

**Q: Which features would you engineer for machine failure prediction?** Rolling statistics of vibration and temperature over several windows, trends (slope over the last hour), time since last maintenance, count of threshold exceedances, operating hours, load, and frequency-domain features from FFT. Each must use only data available before the prediction time.

---

## Chapter 19A: Rapid-Fire Machine Learning Questions

Short answers you should be able to give in 20 to 30 seconds each.

**1. Bias-variance tradeoff in one line?** Simple models underfit (high bias); complex models overfit (high variance). Aim for the balance that minimises validation error.

**2. Why do we need regularization?** To penalise complexity, shrink weights, and reduce overfitting, especially with many features or little data.

**3. L1 vs L2?** L1 can drive weights to zero (feature selection). L2 shrinks all weights smoothly and handles correlated features better.

**4. What is cross-validation for?** A more reliable estimate of generalisation performance and a way to tune hyperparameters without touching the test set.

**5. Bagging vs boosting?** Bagging trains models in parallel on bootstrap samples and averages them (reduces variance). Boosting trains models sequentially, each fixing the previous errors (reduces bias).

**6. Generative vs discriminative models?** Discriminative models learn P(y | x) directly (logistic regression, SVM). Generative models learn P(x | y) and P(y), then apply Bayes (naive Bayes, GMM).

**7. What is an SVM?** A classifier that finds the separating boundary with the largest margin. With kernels it handles non-linear boundaries. Good for medium-sized, high-dimensional data.

**8. What is the kernel trick?** Computing similarities in a high-dimensional feature space without explicitly creating those features, using a kernel function such as RBF.

**9. Why do we shuffle data before training (usually)?** So that mini-batches are representative and gradient updates are not biased by the order of the data. Do not shuffle time series before splitting.

**10. What is a confusion matrix used for?** To see the types of errors a classifier makes, not only its overall accuracy.

**11. What does “stratified” mean?** The split keeps the same class proportions in each subset as in the full dataset.

**12. What is the difference between a model parameter and a feature?** A feature is an input to the model. A parameter is a value the model learns, like a weight.

**13. Why does more data help?** It reduces variance, so the model is less sensitive to noise and to the particular sample. It helps most when the model is complex.

**14. What is ensembling?** Combining several models to get better predictions than any single one. Methods include bagging, boosting and stacking.

**15. What is stacking?** Training several different base models, then training a meta-model on their predictions (made out-of-fold) to combine them.

**16. How do you handle a model that works in testing but fails in production?** Check for leakage in the offline evaluation, compare production input distributions with training data (data drift), check preprocessing parity between training and serving, and verify the label definition.

**17. What is A/B testing for models?** Serving the new model to a random part of traffic and comparing business metrics with the current model. Offline metrics do not always predict online impact.

**18. What is the difference between data drift and concept drift?** Data drift: input distribution changes. Concept drift: the relationship between input and output changes. Both can break a model, and both need monitoring.

**19. Why is interpretability important?** Stakeholders need to trust and act on predictions. It also helps detect bugs, bias, and leakage. Tools include coefficients, feature importance, partial dependence plots, and SHAP.

**20. What is SHAP?** A method that assigns each feature a contribution to a specific prediction, based on game theory (Shapley values). It explains individual predictions and overall feature effects.

**21. What is the difference between precision and recall again, in one sentence each?** Precision: of what I flagged, how much was right. Recall: of what was really positive, how much I found.

**22. What is a learning curve?** A plot of training and validation performance against training set size. If both converge at a low score, the model underfits. If there is a large gap, more data or regularization helps.

**23. How do you explain a model to a non-technical manager?** Start with the business question and the result in plain numbers, show a few examples, explain the main drivers in simple terms, and be clear about the limits and the cost of errors.

**24. What is gradient descent's biggest risk?** Getting stuck in poor local minima or saddle points (more common in high dimensions), or diverging with a bad learning rate. Modern optimisers and initialisation reduce these issues.

**25. What would you check first if a model's score suddenly drops in production?** Data pipeline issues (missing or changed fields, unit changes), input distribution shift, changes in upstream systems, and whether the labels or definitions changed.

---

# Part 5: Domain Topics From the JD

## Chapter 20: Signal Processing

### Big idea

A signal is a quantity that changes over time, like vibration, temperature, or sound. Signal processing cleans it up and extracts useful information.

### Sampling and the Nyquist rule

We record a continuous signal at discrete times (the sampling rate, in Hz).

**Nyquist rule:** To capture a frequency f correctly, the sampling rate must be at least 2f.

**Example:** A machine vibrates at 50 Hz. You must sample at 100 Hz or faster. If you sample at 60 Hz, the 50 Hz vibration appears as a false lower frequency. This is **aliasing**.

### Fourier transform

Converts a signal from the time domain (value over time) to the frequency domain (strength of each frequency). Used to find which frequencies are present.

**Example:** A signal that's a mix of a 5 Hz wave and a 40 Hz wave. The FFT shows two peaks, at 5 Hz and 40 Hz. This is how a machine engineer finds a fault frequency, like a bearing defect.

In Python: `np.fft.fft(signal)`, then look at the magnitude spectrum.

### Filtering

- **Low-pass:** keeps low frequencies, removes high-frequency noise.
- **High-pass:** keeps high frequencies, removes slow drift.
- **Band-pass:** keeps a range, for example 20–200 Hz.
- **Moving average:** replaces each value with the average of its neighbors. Simple low-pass filter.

**Example:** Signal = [1, 2, 6, 2, 1], window = 3. - Position 2: (1 + 2 + 6) / 3 = 3.0 - Position 3: (2 + 6 + 2) / 3 = 3.33 - Position 4: (6 + 2 + 1) / 3 = 3.0 The spike of 6 gets smoothed out.

### Windowing

When you take a finite chunk of a signal for FFT, the sharp edges create false frequencies. A window function (like Hamming or Hann) tapers the edges smoothly to reduce this leakage.

### Signal-to-noise ratio (SNR)

Ratio of useful signal power to noise power. Higher means cleaner signal. Usually shown in decibels: SNR(dB) = 10 × $\log_{10}$(signal power / noise power).

### Interview tip

“How would you detect a fault in a rotating machine from vibration data?” Answer: “I’d sample at a rate above twice the highest frequency of interest, remove noise with a band-pass filter, run an FFT, and look for peaks at the expected fault frequencies, like bearing defect frequencies. Then I’d track those peaks over time.”

### Check yourself

1. What sampling rate do you need to capture a 200 Hz signal? (Answer: at least 400 Hz.)
2. What does an FFT output? (Answer: the strength of each frequency present in the signal.)

---

## Chapter 21: Time Series and Machine Data

### Big idea

Time series data is ordered in time. Each point depends on when it happened, so random splitting and shuffling can break the analysis.

### Components

- **Trend:** long-term direction (going up or down).
- **Seasonality:** repeating pattern at a fixed period (daily, weekly, yearly).
- **Noise:** random leftover.

### Stationarity

A stationary series has a stable mean and variance over time. Many models assume this. Test with the Augmented Dickey-Fuller (ADF) test, or make the series stationary by differencing: $y'_t = y_t - y_{t-1}$.

### Autocorrelation

How much a value is related to earlier values. ACF plot shows this for each lag.

**Example:** Temperature today is strongly related to temperature yesterday (lag 1). Sales this Monday are related to sales last Monday (lag 7).

### Simple forecasting methods

- **Naive forecast:** tomorrow = today. Always compare your model to this baseline.
- **Moving average:** average of the last k values.
- **Exponential smoothing:** recent values get more weight.
- **ARIMA:** combines autoregression (AR), differencing (I), and moving average (MA) terms. Good for stationary-ish series.

### Machine data specifics

- **Sensor data** is often noisy, has gaps, and arrives at uneven times.
- **Resampling:** convert to a fixed frequency, for example 1 minute averages.
- **Rolling windows:** rolling mean and rolling standard deviation over the last N points.

### Anomaly detection

A simple approach: compute a rolling mean and standard deviation, then flag points whose z-score exceeds a threshold (like 3).

**Example:** Vibration readings average 2.0 with std 0.2. A reading of 2.9 has $z = (2.9 - 2.0) / 0.2 = 4.5$. Flag it.

### Predictive maintenance

Predict when a machine will fail, so repairs happen before breakdowns. Typical features: rolling vibration, temperature trends, operating hours, number of past faults. Typical target: failure within the next N days.

### Overall Equipment Effectiveness (OEE)

A common manufacturing metric:

```
OEE = Availability × Performance × Quality
```

**Example:** 90% × 95% × 98% = **83.7%**

### Interview tip

“Why not shuffle time series data before splitting?” Answer: “Shuffling mixes future values into the training set, which leaks information. Always split by time: train on the past, test on the future.”

### Check yourself

1. Why must you split time series by time? (Answer: to avoid training on future data.)
2. What does a z-score of 4 mean? (Answer: the value is 4 standard deviations from the mean, very unusual.)

---

## Chapter 22: Medical Imaging Basics (MRI and CT)

### Big idea

MRI and CT scans produce 3D images of the body. Each image is just a grid of numbers, and data science works with those numbers.

### CT (Computed Tomography)

- Uses X-rays from many angles. A computer reconstructs cross-sections.
- Measures how much tissue absorbs X-rays, expressed in **Hounsfield Units (HU)**.

Typical HU values: | Tissue | Approx. HU | |—|—| | Air | −1000 | | Fat | around −100 | | Water | 0 | | Soft tissue | around +20 to +80 | | Bone | +400 and above |

### MRI (Magnetic Resonance Imaging)

- Uses strong magnets and radio waves. Measures how hydrogen atoms in tissue respond.
- Good for soft tissue (brain, muscles, ligaments). No ionizing radiation.
- Intensity depends on tissue properties and the scan sequence (T1, T2), so the numbers are less standardized than CT.

### DICOM

The standard file format for medical images. Each file contains pixel data plus metadata (patient info, scan settings, slice thickness, pixel spacing). Python libraries: `pydicom`, `SimpleITK`.

### Image as data

- A 2D slice is a matrix of intensities, for example 512 × 512.
- A 3D scan is a stack of slices, a volume.
- Each voxel (3D pixel) has a value.

### Common preprocessing

- **Normalization:** scale intensities to a standard range, for example [0, 1].
- **Resampling:** make voxel spacing the same across scans.
- **Denoising:** Gaussian blur or median filter.
- **Cropping or resizing:** to a fixed input size for a model.
- **Segmentation:** label regions (organs, tumors). Basic methods: thresholding, region growing. Deep learning: U-Net.

### Interview tip

“Why do we normalize CT images?” Answer: “Hounsfield values have a fixed physical meaning, but MRI intensities don’t. Normalization makes scans comparable and helps models train.”

### Check yourself

1. What HU value does water have? (Answer: about 0.)
2. Why is MRI intensity harder to compare across scans than CT? (Answer: it depends on scan settings, not a fixed physical scale.)

---

## Chapter 23: Optical Flow and Computer Vision

### Big idea

Optical flow estimates how each pixel moves between two consecutive frames of a video. The output is a vector field: for each pixel, a motion (u, v) in x and y directions.

### Key assumption: brightness constancy

A point keeps the same brightness as it moves:

```
I(x, y, t) = I(x + u, y + v, t + 1)
```

Using a first-order Taylor approximation, this becomes the **optical flow constraint**:

$$I_x \times u + I_y \times v + I_t = 0$$

where $I_x$, $I_y$ are image gradients in x and y, and $I_t$ is how brightness changes over time.

**Problem:** One equation, two unknowns (u and v). This is called the **aperture problem**.

### Lucas-Kanade method (sparse)

Assume neighboring pixels in a small window move together. Then you get many equations for the same (u, v), and solve them by least squares:

$$\begin{bmatrix} \Sigma I_x^2 & \Sigma I_x I_y \\ \Sigma I_x I_y & \Sigma I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -\Sigma I_x I_t \\ -\Sigma I_y I_t \end{bmatrix}$$

Good for tracking specific points (corners, features).

### Farneback method (dense)

Estimates motion for every pixel by approximating local image regions with polynomials. Gives a full flow field. Good for video analysis.

### Coding example (OpenCV)

```python
import cv2
import numpy as np

cap = cv2.VideoCapture("video.mp4")
ok, frame1 = cap.read()
prev = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)

while True:
    ok, frame2 = cap.read()
    if not ok:
        break
    nxt = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    flow = cv2.calcOpticalFlowFarneback(
        prev, nxt, None,
        pyr_scale=0.5, levels=3, winsize=15,
        iterations=3, poly_n=5, poly_sigma=1.2, flags=0
    )
    magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    print("Average motion:", magnitude.mean())

    prev = nxt
```

### Applying optical flow to medical scans

- Track motion between slices or time frames (for example, breathing or organ motion in dynamic scans).
- Check the image intensity is consistent before applying the brightness constancy assumption. Contrast changes (like contrast agents) break the assumption.
- Preprocess: normalize, denoise, and register (align) frames first.

### Convolution (for CNN and image filters)

A kernel slides over the image. At each position, multiply overlapping values and sum them.

**Example:** Edge detection kernel [−1, 0, 1] applied to row [10, 10, 50, 50]: - At position 2: (−1×10) + (0×10) + (1×50) = 40. Large value = edge.

### Interview tip

“What is the aperture problem?” Answer: “A small window of an edge can only show motion perpendicular to the edge. Motion along the edge is invisible, so we need extra assumptions, like neighboring pixels moving together.”

### Check yourself

1. What does brightness constancy assume? (Answer: a pixel keeps its brightness as it moves.)
2. Which method gives motion for every pixel? (Answer: dense optical flow, like Farneback.)

---

## Chapter 24: Statistical Analysis for Industry Data

### Big idea

Industry data (machines, customers, transactions) is messy. Good analysts check quality first, then describe, then model.

### Typical workflow

1. **Understand the business question.** “Why are defects rising on line 3?”
2. **Check data quality.** Missing values, duplicates, wrong units, timestamps out of order.
3. **Explore.** Summary statistics, histograms, box plots, correlations.
4. **Test hypotheses.** For example: “Defect rate is higher on night shift.”
5. **Model.** Predict or group.
6. **Communicate.** Plain-language findings with a recommended action.

### Outlier and quality checks

- Count missing values per column.
- Check for impossible values (negative weights, dates in the future).
- Compare summary stats across time periods to spot sensor drift.

### Interview tip

Interviewers want to hear the workflow. A common failing answer jumps straight to a model without checking data.

### Check yourself

1. What’s the first thing you check on a new dataset? (Answer: data quality and what each column means.)

---

# Part 6: Tools You Should Know

## Chapter 25: Python, Pandas, NumPy

### NumPy: vectorized math

Instead of looping, operate on whole arrays at once. Faster and cleaner.

```
import numpy as np
x = np.array([1, 2, 3])
print(x * 2)        # [2 4 6]
print(np.mean(x))   # 2.0
print(np.std(x))    # 0.816... (population standard deviation)
```

### Pandas: tables

```
import pandas as pd

df = pd.DataFrame({
    "machine": ["A", "A", "B", "B", "B"],
    "shift":   ["day", "night", "day", "night", "night"],
    "defects": [3, 7, 2, 9, 6]
})

df.groupby("shift")["defects"].mean()    # average defects per shift
df.groupby(["machine", "shift"]).size()  # count rows per group
df.isna().sum()                           # missing values per column
df[df["defects"] > 5]                     # filter rows
```

### Merge and join

```
pd.merge(orders, customers, on="customer_id", how="left")
```

`how="left"` keeps every row from the left table, even without a match.

### Scikit-learn pattern

Every model follows the same pattern:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)          # learn
pred = model.predict(X_test)         # predict
print(classification_report(y_test, pred))
```

### Interview tip

Practice writing these from memory. Interviewers often ask you to write a groupby or a train-test split live.

### Check yourself

1. What does `how="left"` do in a merge? (Answer: keeps all rows from the left table.)

---

## Chapter 26: SQL Basics

### Core query

```
SELECT shift, AVG(defects) AS avg_defects
FROM production
WHERE line = 'L3'
GROUP BY shift
HAVING AVG(defects) > 5
ORDER BY avg_defects DESC;
```

- `WHERE` filters rows before grouping.
- `GROUP BY` groups rows.
- `HAVING` filters groups after aggregation.

### Joins

- `INNER JOIN`: only matching rows.
- `LEFT JOIN`: all left rows, matched right rows where they exist.

### Window functions

Compute something per row using neighboring rows, without collapsing the table.

```
        SELECT machine_id, reading_time, vibration,
               AVG(vibration) OVER (
                   PARTITION BY machine_id
                   ORDER BY reading_time
                   ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
               ) AS rolling_avg_5
        FROM sensor_readings;
```

This computes a rolling average of the last five readings for each machine.

### Interview tip

Know the difference between `GROUP BY` (collapses rows) and window functions (keep rows).

### Check yourself

1. What's the difference between WHERE and HAVING? (Answer: WHERE filters rows, HAVING filters groups.)

---

# Part 7: Putting It Together

## Chapter 27: Answering Interview Questions Clearly

### A simple structure for any "explain" question

1. **One-line definition** in plain words.
2. **Why it matters** (a business or practical reason).
3. **Example** with numbers if possible.
4. **Limitation or when not to use it.**

**Example question:** "Explain random forest." - Definition: "It's many decision trees voting together." - Why: "One tree overfits. Averaging many trees reduces that." - Example: "For predicting machine failure, each tree sees a random part of the sensor data, and the forest's majority vote is more reliable." - Limitation: "It's harder to explain than a single tree and uses more memory."

### Case-study structure

For a question like "How would you reduce defects on a production line?": 1. Clarify the goal and the metric. 2. Check the data available (sensors, logs, labels). 3. Explore and clean. 4. Propose a hypothesis (for example, "temperature drift causes defects"). 5. Choose a model or analysis that fits. 6. Evaluate with a metric that matches the cost of mistakes. 7. Explain how to deploy and monitor it.

### Talking about your projects

- Use the STAR format: Situation, Task, Action, Result.
- State the metric and the baseline.
- Say what you would do differently next time. This shows maturity.

---

## Chapter 28: Practice Questions With Answers

**Q1. What is the difference between variance and standard deviation?** Variance is the average squared distance from the mean. Standard deviation is its square root, which returns the value to the original units.

**Q2. Why is accuracy misleading for fraud detection?** Fraud is rare, so a model that always predicts "not fraud" can be very accurate but catch zero fraud. Use recall, precision, or PR-AUC.

**Q3. What is overfitting and how do you detect it?** The model learns noise in the training data. Detect it when the training score is much higher than the validation score.

**Q4. Explain the difference between bagging and boosting.** Bagging trains models in parallel on bootstrap samples and averages them (random forest). Boosting trains models in sequence, each one correcting the errors of the previous ones (gradient boosting).

**Q5. Why do we need a random forest when we have a decision tree?** A single tree has high variance and overfits. Averaging many decorrelated trees reduces variance.

**Q6. What does the p-value tell you?** The probability of seeing results at least this extreme if the null hypothesis is true. It is not the probability that the null hypothesis is true.

**Q7. How do you choose k in k-means?** Use the elbow method on inertia and the silhouette score. Check that the clusters make business sense.

**Q8. Why do we standardize features before PCA or k-NN?** Both depend on distances or variances. Without scaling, features with large units dominate.

**Q9. What is the Nyquist rate?** The minimum sampling rate, which is twice the highest frequency in the signal.

**Q10. Why can't we shuffle time series before splitting?** Shuffling lets future information leak into training, which makes the test score unrealistically high.

**Q11. What is data leakage? Give an example.** Information from outside the training data gets into the model. Example: fitting the scaler on the full dataset before splitting.

**Q12. Explain gradient descent in one minute.** Start with random weights. Compute the gradient of the loss. Move each weight a small step in the opposite direction, scaled by the learning rate. Repeat until the loss stops decreasing.

**Q13. What does a sigmoid output mean in logistic regression?** A probability between 0 and 1 that the example belongs to the positive class.

**Q14. What is the aperture problem in optical flow?** Within a small window, you can only measure motion perpendicular to an edge. You need extra assumptions, like neighboring pixels moving together.

**Q15. What is a Hounsfield Unit?** A standard scale for CT intensity: water is 0, air is about −1000, and bone is above +400.

---

## Glossary

- **Accuracy:** proportion of correct predictions.
- **Activation function:** a non-linear function applied to a neuron's output.
- **Bias (model):** error from overly simple assumptions.
- **Bootstrap sample:** a sample drawn with replacement from the data.
- **Centroid:** the center point of a cluster.
- **Confusion matrix:** table of true vs predicted classes.
- **Covariance:** how two variables change together.
- **Cross-validation:** repeated train/test splits to estimate performance.
- **Eigenvector:** a direction a matrix only stretches.
- **Epoch:** one full pass through the training data.
- **Feature:** an input variable.
- **Gradient:** vector of partial derivatives.
- **Hyperparameter:** a setting chosen before training, like k or max depth.
- **Label:** the correct answer in supervised learning.
- **Learning rate:** step size in gradient descent.
- **Loss function:** measures how wrong the model is.
- **Nyquist rate:** twice the highest frequency in a signal.
- **Overfitting:** learning noise, poor generalization.
- **p-value:** probability of data this extreme under the null hypothesis.
- **Precision:** of predicted positives, the fraction that are correct.
- **Recall:** of actual positives, the fraction found.
- **Regularization:** penalty that keeps the model simpler.
- **Standard deviation:** typical distance from the mean.
- **Stationarity:** stable statistical properties over time.

- **Variance (model):** sensitivity to the training data.
- **Variance (statistics):** average squared distance from the mean.

---

*These notes cover the material in the job description. Review the chapters you find hardest, and practise explaining each one aloud in your own words.*